

Pandas Data Exploration & Processing

From raw CSV files to understood, cleaned, and validated datasets

Assignment at a glance	Details
Scenario	You've just started a Data Engineer role. Your team has shared real e-commerce data across multiple CSV files.
Dataset	Olist Brazilian E-Commerce Public Dataset (Kaggle)
Tools	Python, Pandas, JupyterLab, VS Code, Git/GitHub
Workflow	Notebook → Python Script → Notebook
Scope	Explore individual files, make reasoned cleaning decisions, process files, and validate results. No database or complex joins in this assignment.

A note from Uma Kiran: This assignment is not about rushing to produce a perfectly clean dataset. It is about learning how to inspect unfamiliar data, ask good questions, make defensible decisions, and prove that your code did what you intended.

1. Why are we doing this assignment?

Congratulations on your new Data Engineer role! You've joined the data team at a growing e-commerce company. Your team works with business stakeholders who rely on accurate, well-understood data to answer questions about orders, customers, products, and customer experience.

Your team has now handed you your first mini project.

The business situation

The company has collected data from its online store, but the information is spread across several CSV files. Before analysts can confidently use it for reporting and business insights, the team needs to understand what the files contain and whether the data is suitable for use. You have been asked to take the first step: inspect the files, identify potential data-quality concerns, apply carefully considered processing, and validate the results.

Imagine your team lead says:

"We've received these e-commerce data files and need to understand what we're working with. Please explore the data, document anything that may affect its quality or usability, and prepare processed versions where appropriate. Don't change the original files. We need to be able to understand what you did and trust the results."

Your mini-project objective

Your task is to move from unfamiliar raw data to data that has been explored, thoughtfully processed, and validated. You are not expected to know every business rule in advance. Your job is to investigate, ask questions, record assumptions, and make decisions you can explain.

Why this matters

In real Data Engineering work, data rarely arrives perfectly organized and ready to use. It may be spread across files, contain missing values, inconsistent formats, duplicate records, or values that need business context before they can be changed. Downstream reports and applications can be affected when these issues go unnoticed.

This mini project will help you build the habits expected of a Data Engineer:

- Use Pandas to read and inspect unfamiliar CSV files.
- Understand rows, columns, data types, distributions, missing values, duplicates, and categories.
- Record observations and investigate before making cleaning decisions.
- Turn your decisions into a repeatable Python script.
- Validate the processed output instead of assuming the code worked.
- Use Git to track your work and communicate your reasoning.

The goal is not to run as many Pandas commands as possible. The goal is to understand the data, make reasoned decisions, and produce results you can explain and verify.

2. Your scenario and dataset

As a Data Engineer, Your first task is to understand a dataset from an online retail business. The data is shared as separate CSV files. You have not been given a data dictionary or cleaning instructions—so you must investigate carefully.

Dataset page:

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Kaggle is an online platform where people explore datasets, learn data science and machine learning, and work on projects or competitions. It also hosts public datasets shared for learning, research, and analysis.

If you do not already have a Kaggle account, create one or sign in before downloading the dataset.

Required files

File	Starting questions
orders.csv	What does one row appear to represent? Which columns describe order timing or status?
customers.csv	What customer-related attributes are available? Are identifiers present?
order_items.csv	What item-level information is recorded? What do price and freight values look like?
products.csv	Which product attributes are populated? Are categories consistent?
order_reviews.csv	What review-related fields exist? Are scores or text values missing?

Download these five CSV files and place them in `data/raw/`. You may explore other Olist files, but they are not required.

Important: Treat `data/raw/` as immutable. Do not edit, clean, rename columns in, or overwrite the original files. Your processing must read from raw and write separate outputs to `data/processed/`.

3. What you will build

You will complete three connected parts. Each part has a different purpose; do not skip directly to writing a large script.

Part	Where you work	Purpose	Expected outcome
A — Explore	Jupyter notebook	Understand each file and document potential quality issues before deciding what to do.	Exploration notebook + docs/observations.md
B — Process	Python script in VS Code	Implement your documented decisions in repeatable code.	Script(s) + processed CSV files
C — Validate	Jupyter notebook	Check whether processing produced the intended results and preserved important data.	Validation notebook + recorded findings

4. Project setup

4.1 Create your project structure

Create a GitHub repository for this assignment, clone it to your computer, and open the project folder in VS Code. Create the following structure:

```
olist-pandas-assignment/
├── data/
│   ├── raw/
│   └── processed/
├── src/
├── notebooks/
├── docs/
│   └── observations.md
├── README.md
├── requirements.txt
└── .gitignore
```

Keep notebooks in notebooks/, reusable processing code in src/, your observations in docs/, and the original/processed CSVs in their respective folders.

4.2 Create and activate a virtual environment

Run these commands from the project root in the VS Code terminal:

```
python -m venv .venv
```

Activate it using the command for your operating system:

Operating system / terminal	Command
Windows Command Prompt	.venv\Scripts\activate
Windows PowerShell	.venv\Scripts\Activate.ps1
macOS / Linux	source .venv/bin/activate

When activated, your terminal prompt should show (.venv). If activation is blocked in PowerShell, do not randomly change system settings—ask for help or use Command Prompt.

4.3 Install the required packages

```
python -m pip install --upgrade pip
python -m pip install pandas jupyterlab ipykernel
python -m pip freeze > requirements.txt
```

The virtual environment keeps this project's Python packages separate from other projects. The requirements.txt file records the packages installed for this assignment.

4.4 Launch JupyterLab — your notebook workspace

For this assignment, use JupyterLab as your primary notebook interface. It runs in your browser but uses the Python environment on your computer. Launch it from the activated project environment:

```
jupyter lab
```

- JupyterLab should open in your browser. If it does not, copy the local URL shown in the terminal and paste it into your browser.
- Keep the terminal window running while you use JupyterLab.
- Navigate to the project's notebooks/ folder and create a Python notebook there.
- Select the kernel associated with this project's .venv environment.
- Use Shift + Enter to run a cell. Use Markdown cells to explain what you are doing and what you observe.
- When finished, stop JupyterLab with Ctrl + C in the terminal and confirm if prompted.

If the project environment does not appear as a kernel, make sure .venv is activated and run:

```
python -m ipykernel install --user --name olist-pandas --display-name "Python (olist-pandas)"
```

JupyterLab, VS Code, and Databricks — what is the difference?

Tool	How you will use it
JupyterLab	Your main notebook workspace for Part A exploration and Part C validation. It is browser-based and runs locally.
VS Code	Your general-purpose editor for Python scripts, project files, and Git. It is where you will primarily write and run Part B scripts.
VS Code notebooks (optional)	VS Code can also open notebooks if the Microsoft Python and Jupyter extensions are installed. This is an alternative—not a second setup you must learn.
Databricks	A cloud data platform that also uses notebooks, but adds managed compute and data-platform capabilities. The notebook concepts you learn here will transfer; the environment and platform features differ.

Pandas is the same library regardless of whether you use JupyterLab or VS Code. The difference is the working interface and workflow. Do not install or learn multiple notebook interfaces just for this assignment.

4.5 Git and raw data safety

Add at least the following to .gitignore:

```
.venv/  
__pycache__/  
.ipynb_checkpoints/
```

Do not commit your .venv folder. Follow your instructor's repository/data-size guidance for the downloaded CSVs. Never overwrite raw data. Commit your work in small, meaningful steps.

```
git status  
git add .  
git commit -m "Set up Pandas assignment project"  
git push
```

5. Part A — Explore and understand the data (Notebook)

Create a notebook such as notebooks/01_data_exploration.ipynb. Work through each required CSV one at a time. The goal is to understand the file—not to clean it immediately.

For every file, create a short Markdown heading and explain the question you are investigating. Run an operation, inspect the output, and write down what you learned. Do not run every command without looking at the results.

5.1 Load each CSV

Use Pandas to load one file into a DataFrame. A DataFrame is a table-like structure with rows and columns.

```
import pandas as pd  
  
orders = pd.read_csv("../data/raw/olist_orders_dataset.csv")
```

Adjust the filename and path to match the exact downloaded file. Repeat for each required CSV, using a clear variable name. If your notebook is in notebooks/, paths such as ../data/raw/... move up one folder before entering data/raw/.

Purpose: Confirm that Pandas can read the file and that you are working with the intended dataset. If you get a file-not-found error, inspect your folder structure and filename rather than guessing.

5.2 Inspect a few records — head() and tail()

Operation	What it shows / why it matters	Your task
df.head()	Displays the first five rows by default. Helps you see column values and the general shape of records.	Run it. Identify what a row seems to represent and note anything surprising.
df.head(10)	Displays the first 10 rows. Useful when five rows are not enough to see variety.	Try a larger number and compare.
df.tail()	Displays the last five rows by default. Can reveal patterns or unusual records near the end of the file.	Run it. Do the last rows look structurally similar to the first rows?

Do not assume the first or last rows represent the whole dataset. These are quick samples, not a complete quality check.

5.3 Understand the size — shape

Use df.shape. It returns a pair: (number of rows, number of columns).

Purpose: Understand the scale of the file and establish a baseline before processing. Record the row and column counts for each required file. Later, compare them with the processed output and investigate any unexpected change.

5.4 Inspect columns and data types — info()

Use df.info(). It summarizes column names, non-null counts, data types, and memory usage.

- Compare each column's non-null count with the total number of rows.
- Notice columns stored as integers, decimals, text (object), dates, or other types.
- Ask whether a type appears suitable for the values and how the column may be used.
- Flag columns that may need closer inspection—for example, date-like values stored as text.

Do not convert every column just because you can. Record what you observe and why a type might matter.

5.5 Review summary statistics — describe()

Use df.describe() for a quick statistical summary of numeric columns. It commonly includes count, mean, standard deviation, minimum, quartiles, and maximum.

- Check whether minimum or maximum values look unexpected for the field.
- Compare the mean and median-like 50% value to notice skew or unusual ranges.
- Look for impossible-looking or suspicious values, but do not label them wrong without understanding the field.
- Remember: by default, describe() focuses on numeric columns. You may explore categorical summaries separately.

Record questions raised by the summary. A surprising value is a reason to investigate—not automatic proof that the value should be removed.

5.6 Check missing values — `isna().sum()`

Run `df.isna().sum()`. This counts missing values in each column.

Purpose: Missingness can affect analysis and downstream processing. Identify which columns contain missing values and how many. Then ask: Is the field required? Could missingness be expected? Is there another field or business rule that explains it?

```
orders.isna().sum()
```

Do not automatically drop every row with a missing value or fill every blank with zero. Different columns have different meanings; document a proposed action and its rationale.

5.7 Check potential duplicate rows — `duplicated()`

Run `df.duplicated().sum()` to count rows that Pandas considers duplicates across all columns (by default, keeping the first occurrence).

Purpose: Duplicate records can inflate counts or distort analysis. But identical-looking rows are not always duplicates in the business sense, and repeated identifiers may be valid in item-level data. Note the count, inspect examples if any exist, and explain what you need to verify before removing anything.

```
orders.duplicated().sum()
```

5.8 Explore distinct values and categories

Operation	Purpose	Questions to investigate
<code>df['column'].unique()</code>	Shows distinct values in a column.	What values exist? Are there spelling/case/format differences? Are unexpected values present?
<code>df['column'].nunique()</code>	Counts distinct values (missing values excluded by default).	Is the number of unique values reasonable for this type of field?
<code>df['column'].value_counts()</code>	Counts how often each value appears.	Which categories are common or rare? Are there unusual labels or unexpected distributions?

Replace 'column' with an actual column name from your DataFrame. Use these on relevant categorical or identifier columns, not blindly on every column. If a column has many unique values, inspect a manageable subset or use `value_counts()` with an appropriate limit.

```
orders["order_status"].unique()
orders["order_status"].nunique()
orders["order_status"].value_counts()
```

The example column is illustrative—confirm the exact column name in your own file. Do not assume a category is invalid simply because it is rare.

5.9 Select and inspect specific rows — `.loc[]` and `.iloc[]`

Sometimes a summary tells you that something needs a closer look. Use row selection to inspect specific records.

Selection	How it works	Example
-----------	--------------	---------

<code>.loc[]</code>	Selects by index label and/or named column labels. It is label-based.	<code>df.loc[0, 'order_status']</code>
<code>.iloc[]</code>	Selects by integer position (zero-based row/column positions). It is position-based.	<code>df.iloc[0, 2]</code>

Examples:

```
# Select the row whose index label is 0
orders.loc[0]

# Select a row by its integer position
orders.iloc[0]

# Select specific columns by their names
orders.loc[0, ["order_status"]]

# Select rows by a condition (replace with a real condition)
orders.loc[orders["order_status"] == "delivered"]
```

Use these examples to inspect records that relate to an observation. Index labels and row positions may differ, especially after filtering or resetting an index. Always confirm what you selected.

Reflection: Which approach uses labels? Which uses positions? Why might selecting by a meaningful condition be more useful than selecting row 0?

5.10 Maintain your observations log

Create docs/observations.md. For each required CSV, record:

- File name and initial row/column count.
- Important columns and data types you noticed.
- Missing-value counts and where they occur.
- Potential duplicate findings.
- Interesting distinct values, category counts, or unusual distributions.
- Questions or risks that need more investigation.
- Your proposed cleaning/processing decisions and the reason for each.

Use this template for each finding:

```
### File: [file name]
- Observation:
- Evidence (operation/output):
- Why it may matter:
- Proposed action (or no action):
- Reason / assumption to verify:
```

At this stage, observations and proposals are enough. You are not expected to know every business rule. Be transparent about assumptions and uncertainty.

6. Part B — Implement processing in Python scripts

Once you have explored the files and recorded your decisions, move from interactive investigation to repeatable processing. Create a script in `src/` (for example, `src/process_olist_data.py`).

A notebook is excellent for exploration because you can run small cells, inspect outputs, and add notes. A Python script is better for repeatable execution: you can run the same sequence again, process files consistently, and keep the logic organized.

Your implementation steps

1. Read the relevant raw CSV file(s) using Pandas.
2. Apply only the transformations you proposed in docs/observations.md.
3. Keep the code readable. Use clear variable names and small, understandable steps.
4. Save each processed DataFrame as a separate CSV under data/processed/.
5. Print or log useful completion information, such as the output path and resulting shape.
6. Run the script from the project root and investigate any errors.

You decide the appropriate cleaning actions based on your exploration. Do not apply a rule simply because it appears in a tutorial. Preserve raw files and make your processing reproducible.

Useful concepts to research and apply only when relevant: selecting columns, filtering rows, handling missing values, removing confirmed duplicate rows, renaming columns, converting data types, and writing CSV files.

Save processed files with clear names that distinguish them from the raw versions. Do not introduce database schemas, PostgreSQL, complex joins, or a full orchestration pipeline in this assignment; those topics belong to a later stage.

Run your script

```
python src/process_olist_data.py
```

If you encounter an error: read the full message, identify the line, inspect the relevant variable or file path, make one focused change, rerun, and verify. Avoid changing several things at once.

7. Part C – Validate the processed data (Notebook)

Create notebooks/02_validate_processed_data.ipynb. Validation is not optional. A script completing without an error does not prove the output is correct.

Load the processed CSV files into Pandas and compare them with the corresponding raw files.

Validation checklist

- Can Pandas read every processed file?
- Do the processed row and column counts match your expectations? If they changed, can you explain why?
- Are the intended columns present, and are their data types suitable?
- Did the missing-value situation change as intended?
- Are confirmed duplicates handled according to your documented decision?
- Do distinct/category values look consistent with the transformations you intended?
- Can you inspect sample rows to confirm the output is sensible?
- Did the script leave the original raw files untouched?

Record validation evidence and any remaining issues in the notebook or observations.md. If something is wrong, return to the script, fix it, rerun, and validate again.

8. Notebook vs. Python script – understand the difference

Aspect	Jupyter Notebook	Python Script
--------	------------------	---------------

Primary use here	Explore, experiment, inspect, explain, validate	Repeat processing consistently
Execution	Run selected cells, often in an interactive order	Run the program as a sequence
Output	Code, tables, charts, and Markdown notes together	Program output/logs; results can be saved to files
Strength	Fast feedback and visible reasoning	Repeatability, maintainability, automation-readiness
Risk to watch	Hidden state or cells run out of order	A script may run successfully but still produce incorrect results
Assignment part	A and C	B

Neither replaces the other. Use the notebook to understand and validate; use the script to repeat the processing logic.

9. Git checkpoints and submission

Commit in small, meaningful increments rather than waiting until the end. Example commit messages:

- Create project structure and environment files
- Add initial dataset exploration notebook
- Document data quality observations
- Implement CSV processing script
- Add processed data validation notebook

Before submitting, check your repository and README. Explain how to set up the environment, where the raw files belong, how to launch the notebook, how to run the script, and what the assignment contains.

Submission checklist

- ☐ GitHub repository with the agreed project structure
- ☐ requirements.txt and .gitignore
- ☐ Raw CSVs kept separate and unmodified (follow instructor guidance on committing dataset files)
- ☐ Part A exploration notebook with explanations and inspected outputs
- ☐ docs/observations.md with evidence, questions, and reasoned decisions
- ☐ Part B Python processing script(s)
- ☐ Processed CSV files in data/processed/
- ☐ Part C validation notebook with checks and findings
- ☐ README.md with setup and execution instructions
- ☐ Meaningful Git commits pushed to GitHub

10. DataKern engineering principles

Principle	What it means in this assignment
-----------	----------------------------------

Understand before you transform	Inspect the data and record evidence before changing it.
Reason over recipes	A cleaning command is not a decision. Explain why the action is appropriate.
Build → Run → Inspect → Fix → Commit	Work incrementally. Avoid writing everything at once and hoping it works.
Make assumptions visible	If you do not know a business rule, say so. Do not disguise a guess as a fact.
Raw data is sacred	Keep source files unchanged so processing can be repeated and audited.
Validate, don't assume	Check the output against your intent and document the evidence.
Learn by doing the thinking	Use documentation and error messages to build understanding, not to copy blindly.

Important learning rule

Do not use AI code-generation or coding-agent tools to produce your assignment solution. This includes ChatGPT, Gemini, Claude, GitHub Copilot, Cursor, or similar tools. You may use Google, official documentation, tutorials, Stack Overflow, and error messages to research concepts and understand errors—but you must write, explain, and validate your own solution.

During review, you should be able to explain what your code does, why you chose each transformation, what you tested, and what you learned.

Don't focus on getting the answer. Focus on learning how to engineer the solution.

— Uma Kiran | DataKern